

Exercise #4 – Continuous Transformations for Scene Understanding

Mason McGaffin

Exercise #4

ECEN 5763 Embedded Computer Vision

University of Colorado Boulder

June 03, 2026

Contents

1 USE OF THE HOUGH TRANSFORMATION TO DETECT LINES AND CURVES IN PICTURES	3
2 HOUGH LINES – LANE DETECTION	4
3 SKELETAL TRANSFORM	9
4 SCALE-INVARIANT FEATURE TRANSFORM	13
4.1 SUMMARY	13
4.2 MATLAB EXAMPLES	14
4.3 C EXAMPLES	15
4.4 SIFT vs ORB	20
4.5 PYRAMIDAL TRANSFORM	21
OPTION #4 – DEEPER DIVE ON CUDA.....	23
5 BASIC CUDA	23
5.1 VECTORADD.....	23
5.2 MATRIXMUL	24
5.3 BANDWIDTHTEST	25
5.4 SIMPLECUDAGRAPHS.....	27
6 CUDA INTERACTIVE HOUGH.....	28
6.1 STATIC IMAGE.....	28
6.2 LIVE WEBCAM.....	31
7 REFERENCES.....	35
8 APPENDIX	36

1 Use of the Hough Transformation to Detect Lines and Curves in Pictures

Read the paper Use of the Hough Transformation to Detect Lines and Curves in Pictures, Richard Duda & Peter Hart (also available on Canvas) and summarize the papers key points and contributions to computer vision.

The paper, *Use of the Hough Transformation to Detect Lines and Curves in Pictures* [2], by Duda & Hart, is an extension of Hough's original slope-intercept parameterization used to identify lines and curves in images.

The first key point is that the model can be improved by using a normal parameterization (1) instead of the original slope-intercept parameterization. The authors identify the unboundedness problem of the original model, and present the normal parameterization as a solution to reduce the parameter space by representing lines by the angle, θ , of their normal and the distance, ρ , perpendicular to their origin.

$$\rho = x \cos \theta + y \sin \theta \quad (1)$$

The second key point is that the model can be implemented using a 2D array of accumulators which improves performance. For each point in the image, array bins are incremented by tracing the curve from (1). Bins with high counts represent lines in the image. The authors demonstrate that this method has a linear computational growth factor as opposed to the original Hough detection which grows quadratically.

Lastly, the paper discusses some examples and generalizations of the improved algorithm. The authors claim that the approach can be applied to any parametric curve and demonstrate so with the example of circles, which follow (2). It is evident that this paper describes the algorithms we have used so far in this course with OpenCV's HoughLines and HoughCircles APIs.

$$(x - a)^2 + (y - b)^2 = c^2 \quad (2)$$

2 Hough Lines – Lane Detection

*Build and run the basic Hough lines example provided by OpenCV and adapt it for use with a camera or video source with the ability to specify the source as camera or video on the command line. Determine the best method (*HoughLines*, *HoughLinesP*) and parameters to use to find the edges of a roadway from the one of the following challenge set videos – *Self-Driving-Car-Video-1*, *Self-Driving-Car-Video-2*, or *Colorado-Challenge-Set*. Paste a transformed image showing your best detection of roadway edges and lanes in your report.*

As mentioned in Exercise #3, *HoughLinesP* is a better algorithm for most real-world applications, including lane detection, because it uses line length and gap parameters. The method is optimized to measure line lengths, not infinite lengths. However, in the case of predicting straight line paths or angle calculation, the *HoughLines* API can be preferred.

In order to determine the best parameters to use for lane and roadway edge detection, I first utilized my Interactive Hough Detection application from Exercise #3, on a static frame (frame 100) of [Self-Driving-Car-Video-1-Raw/22400003.avi](#), by dragging the trackbars on the static image. This was a good starting point, but I also implemented trackbars, later commented out, on the live video feed as well of the `hough_lines_p` application. The goal was to analyze how each parameter, listed in the Usage below, affected the lane detection. Eventually, I found the following parameters to be best:

Canny:

```
lowThreshold = 30
highThreshold = 100
```

Hough:

```
rho = 1
theta = CV_PI/180
threshold = 15
length = 10
gap = 10
```

```
Usage: hough_lines_p [params] input

-C, --cannyHigh (value:100)
    Canny high threshold
-T, --theta (value:<CV_PI/180>)
    Hough line theta
-c, --cannyLow (value:30)
    Canny low threshold
-g, --gap (value:10)
    Hough line gap
-h, --help (value:true)
    show help message
-l, --length (value:10)
    Hough line length
-r, --rho (value:1)
    Hough line rho
-t, --threshold (value:15)
    Hough threshold
-w, --webcam (value:false)
    live video feed using webcam

input (value:video/02260001_copy.mp4)
    input video
```

Code 2.1 hough_lines_p usage

The user can specify parameters for the application using the command line. If they specify the live video feed option, a live interactive window will appear to see the transformation in real-time. If not, the application runs headless, writing a file called *hough_<input_file>.mp4*. When run using the parameters calculated before, I observed fairly accurate lane detection in simple environments as seen in Figure 2.1, but only occasional success in more challenging videos such as in Figure 2.3, 2.4, 2.5, 2.6. This was primarily due to the color saturation of the roadway lines. In easier settings, edges are easily detected creating a very clear road shape, as seen in Figure 2.2. When the lanes are not as bright, the algorithm has a tough time identifying them. Also, a large factor during development was interference from trees, sidewalk activity, the sky, etc. In order to account for this, a region of interest was implemented to reduce the lane detection to the region of the frame most likely to show the lanes. Because the lane detection algorithm struggled to identify the entire lane, only identifying bits and pieces, a method to filter, sort, and average the lines detected was used to draw single lane lines as seen in the figures below.



Figure 2.1 Lane Detection on video/02260001_copy.mp4 – Easier environment



Figure 2.2 Edge detection on video/02260001_copy.mp4

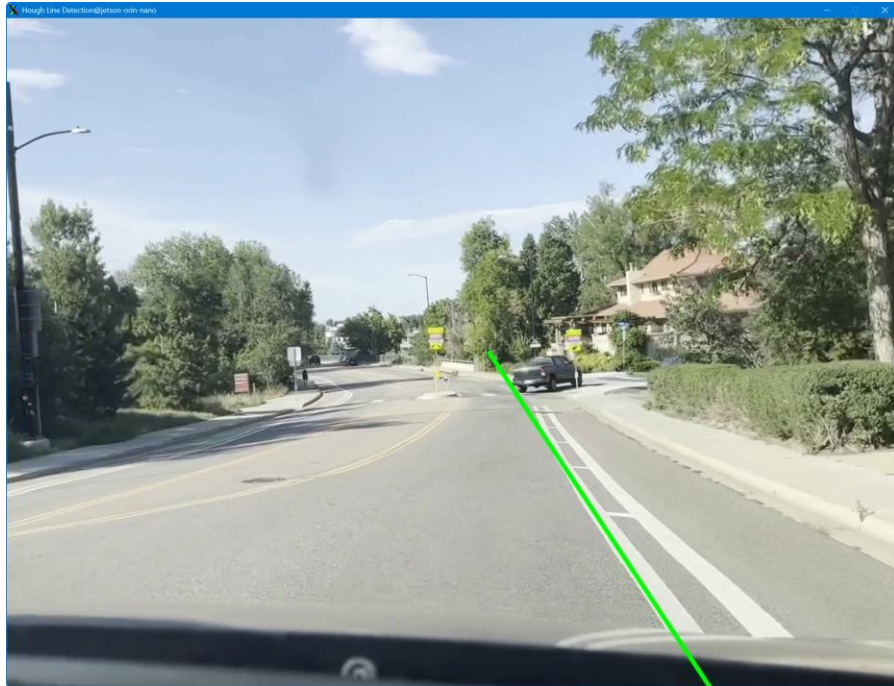


Figure 2.3 Lane detection failure on video/IMG_2852_short.mp4 – challenging environment

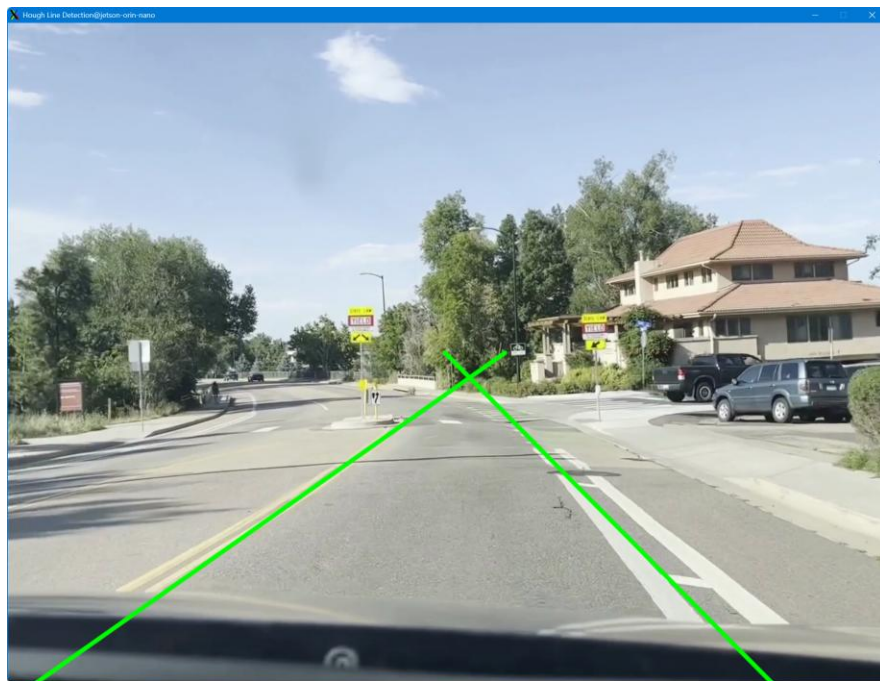


Figure 2.4 Lane detection success on video/IMG_2852_short.mp4 – challenging environment

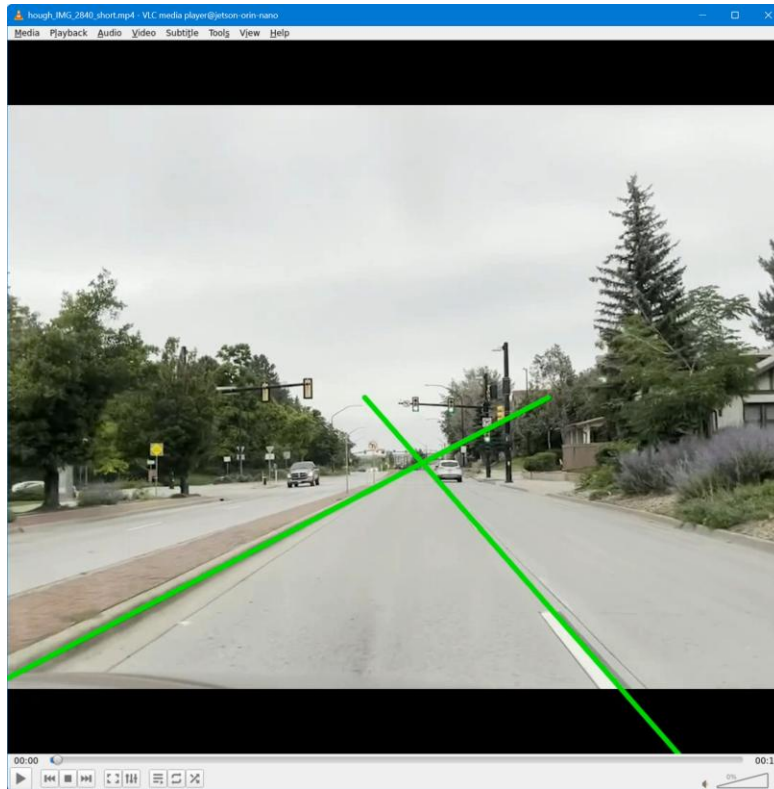


Figure 2.5 Lane detection on video/IMG_2840_short.mp4 - challenging environment

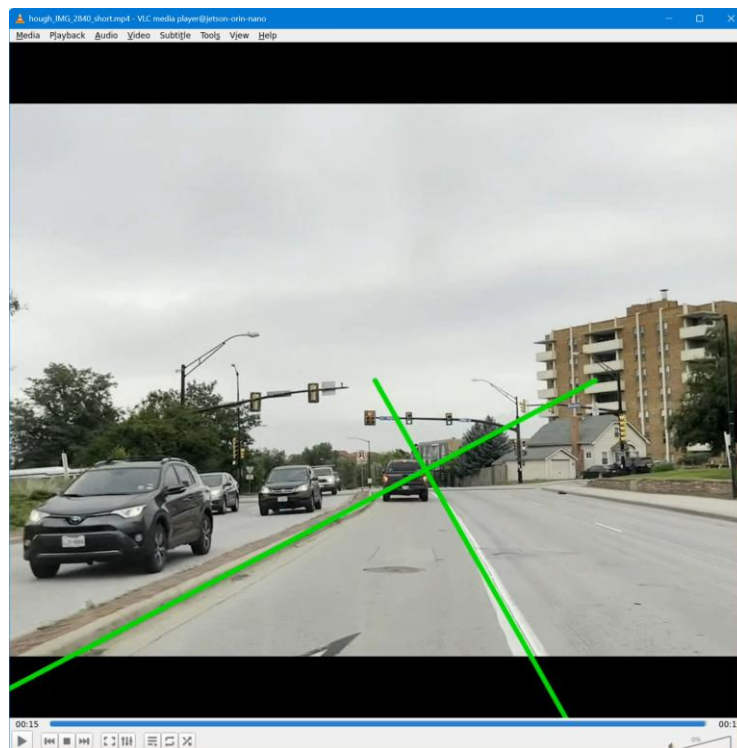


Figure 2.6 Lane detection on video/IMG_2840_short.mp4 – turn lane

3 Skeletal Transform

Using a top-down OpenCV approach, adapt the example code for OpenCV version used in class based on examples found in `capture_transformer` to use the `skeletal.cpp` transform on continuous frames (like `captureskel.cpp`) from your camera, but use the much simpler approach of `simpler-capture` rather than V4L2 camera capture. Gesture in front of your camera and see if you can get a reasonable continuous skeletal transform of your arm and hand, holding up 1, 2, 3, 4, or 5 fingers to see if you could distinguish each and count fingers held up. Record example frames (at least 300 frame and no more than 3000 for 10 to 100 seconds of video – JPEG frames are fine) and encode your results to an MPEG video. Upload the modified code with your report.

The skeletal application successfully identifies the hand in the image, and draws a skeletal trace of the limb. Figure 3.1 shows the intermediate steps of the input image in grayscale, the binary threshold conversion, and the median blur applied. As seen in the figure, the application correctly shows a hand with five fingers. For further demonstration, the video, “skeletal_webcam.mp4”, shows the application correctly identifying zero to five fingers held up. Figures 3.2-3.7 showcase this.

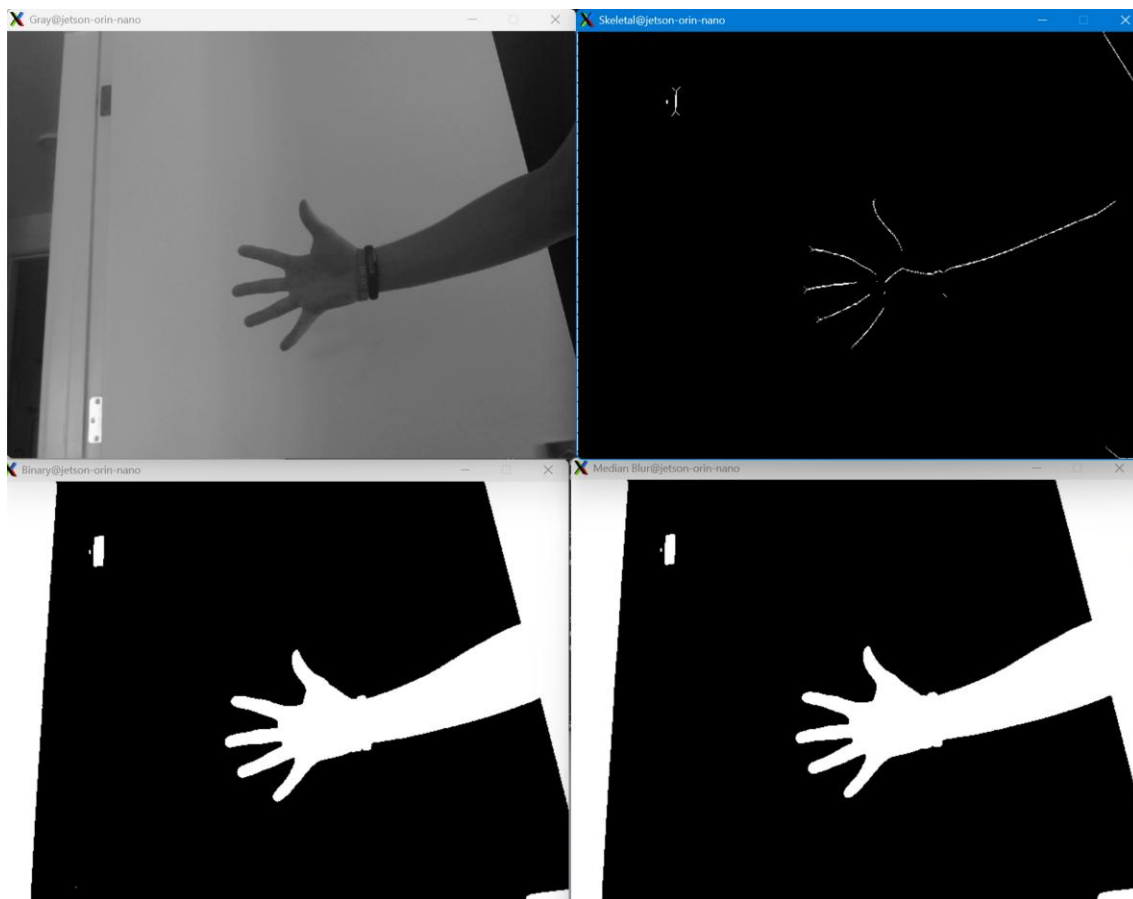


Figure 3.1 Debug of skeletal application

Despite multiple implementation attempts, I was unable to develop a reliable finger-counting algorithm within the assignment timeframe. I consulted artificial intelligence with no success either. I was more concerned with finishing the rest of the assignment then spending hours debugging this issue. There were many difficulties to this problem. The first was that the skeletal transform was not perfectly one-pixel lines. It had sections with clusters of pixels and many breaks and turns. The combination of these and noise from the background made it difficult to identify whole fingers without counting them multiple times. Instead, I focused on getting a clear skeletal image as seen in the examples below

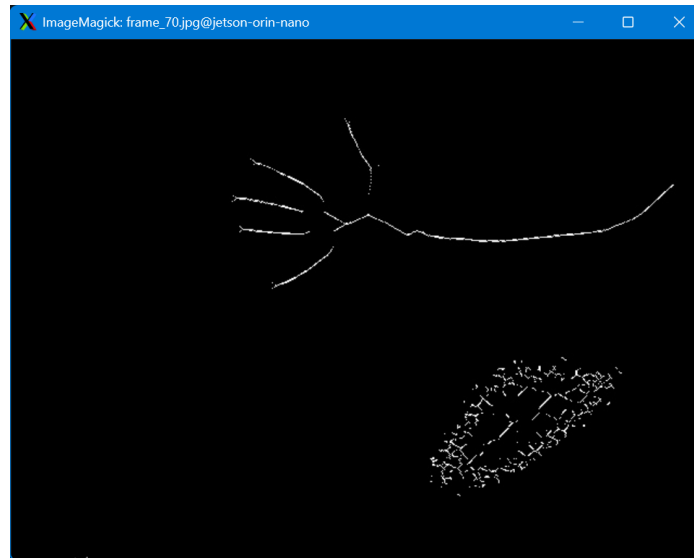


Figure 3.2 skeletal transform – five fingers held

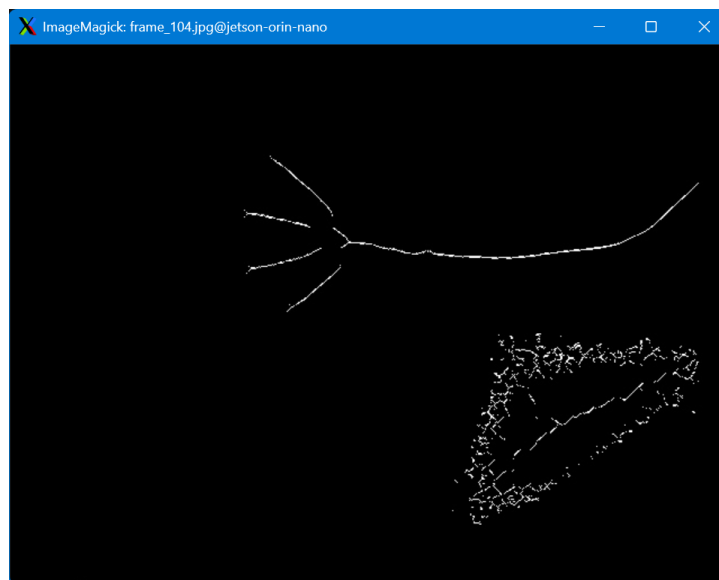


Figure 3.3 skeletal transform – four fingers held



Figure 3.4 skeletal transform – three fingers held – “OK” sign

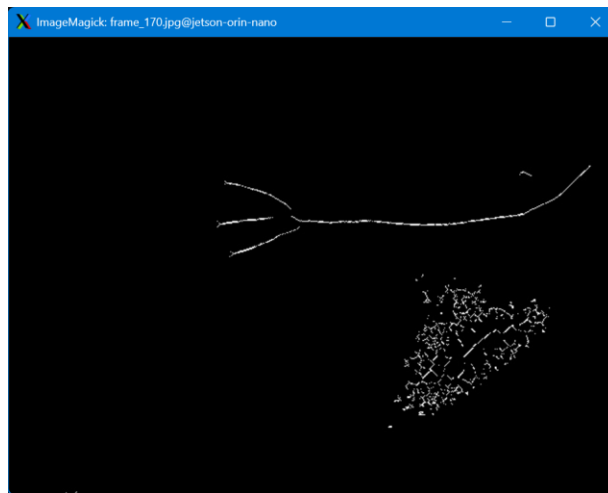


Figure 3.5 skeletal transform – three fingers held – middle three fingers

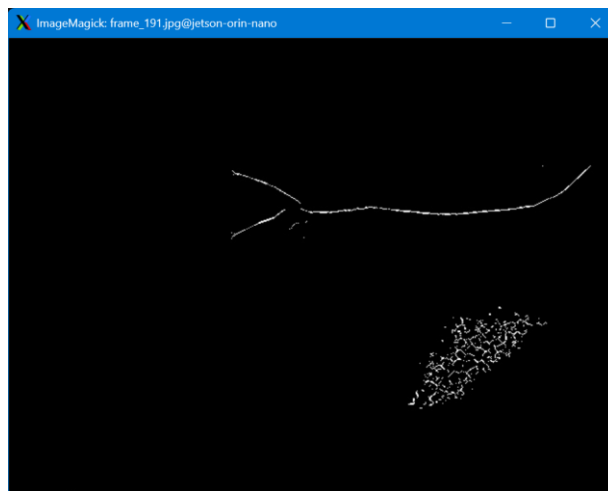


Figure 3.6 skeletal transform – two fingers held



Figure 3.7 skeletal transform – one finger held

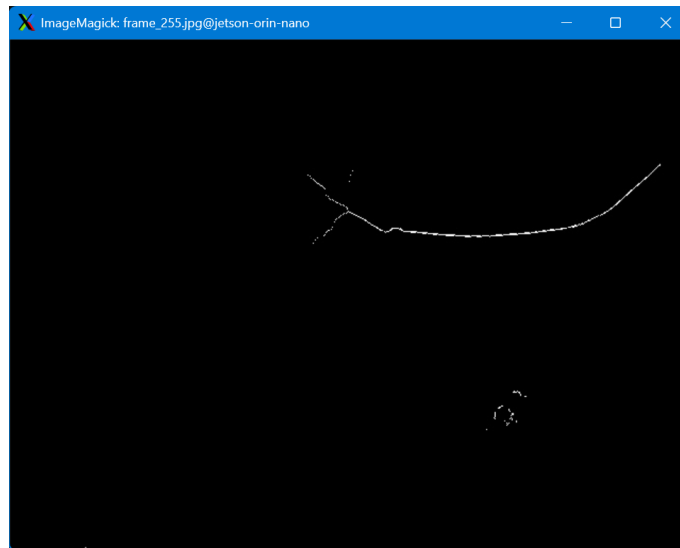


Figure 3.8 skeletal transform – no fingers held

As seen in the figures above, when one through five fingers are held up the transform easily captures the fingers. However, when no fingers are held up, the outline of some fingers in a closed fist is still identified. The shadows create an edge so the transform identifies this part of the palm as a Hough line.

4 Scale-Invariant Feature Transform

Read the paper [Distinctive Image Features from Scale-Invariant Keypoints](#), by David Lowe (also available at [code/siftDemoV4/](#)) and summarize the papers key points and contributions to computer vision. Download the sample code here - <https://www.cs.ubc.ca/~lowe/keypoints/siftDemoV4/>) and run the demos and show that you have done this by incorporating a screen shot of correspondence between book and scene and providing a brief explanation – create keys in Windows or MATLAB. Compare SIFT to a newer OpenCV method called ORB (https://docs.opencv.org/4.1.1/d1/d89/tutorial_py_orb.html) and give at least 3 advantages of which method you believe is best (SIFT or ORB). Finally, in SIFT, the Pyramidal transform (https://docs.opencv.org/4.1.1/d4/d1f/tutorial_pyramids.html) is a key part of the overall SIFT algorithm, so please download PyrUpDown code and describe what happens if you zoom in and then zoom out at the default level of zoom several times, returning to the original zoom level and explain what happens.

4.1 Summary

The paper, *Distinctive Image Features from Scale-Invariant Keypoints* [3], by David G. Lowe, introduces the Scale Invariant Feature Transform (SIFT). This algorithm detects, describes, and matches image features on images that can be scaled, rotated, or similarly modified.

The first key point of the paper is that stable keypoints can be detected across image scales using the Difference-of-Gaussian (DoG) function. The SIFT algorithm uses a scale-space pyramid at increasing blur levels, then subtracts adjacent layers to get the DoG images. Keypoints are found using local extrema compared to their 26 neighbors across DoG layers. From there, the algorithm applies a Hessian matrix, which isolates corners and other keypoints that repeat reliably under image transformation.

The second key point is the SIFT descriptor, which allows the features to be matched across transformed images. The algorithm computes a 16x16 gradient orientation around each keypoint, translating each into sixteen (16) eight-bin histograms, represented as a 128-element vector. The keypoint is assigned a dominant orientation to create rotation invariance. Next, the algorithm accounts for sensitivity to lighting by normalizing and clamping the vector. The resulting descriptor is very unique and can be identified under viewpoint changes and significant noise.

Lastly, the paper discusses object recognition pipeline. Keypoints are matched to a training set using nearest-neighbor search. False matches are filtered out by comparing ratios of closest and second-closest neighbors. If they pass filtering, clusters of matches are identified using a 4D Hough Transform over position, scale, and orientation. Clusters are then verified with affine fitting and outlier rejection. The paper shows the SIFT algorithm can recognize objects reliably even in cluttered scenes. The paper is a great contribution to embedded computer vision as it works in the real world, allowing for object detection even with varying viewpoints, lighting, scale, and orientation.

4.2 MATLAB Examples

```
[img, desc, locs] = sift('book.pgm'); // replace with 'box.pgm'  
showkeys(img, locs);
```

Code 4.1 MATLAB command to sift and show feature descriptors

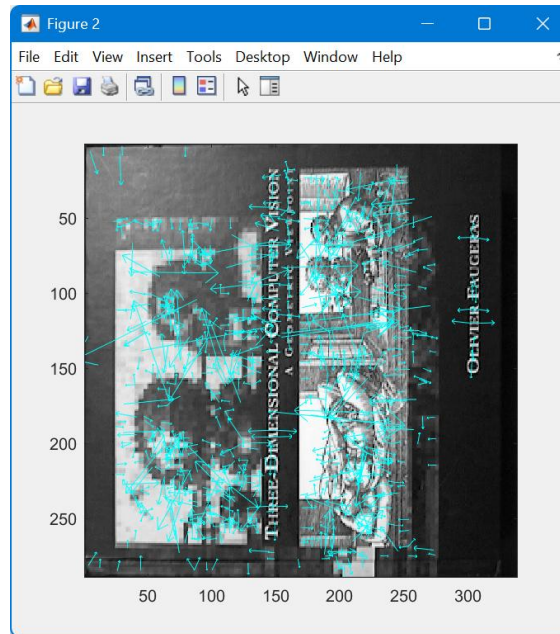


Figure 4.1 showkeys on sifted book.pgm

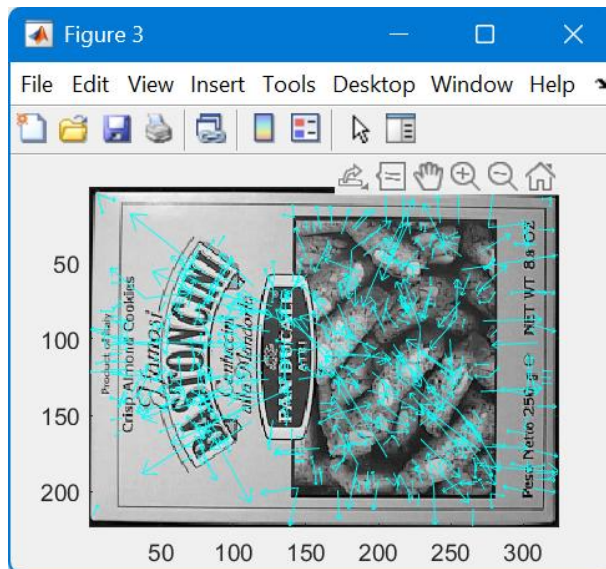


Figure 4.2 showkeys on sifted box.pgm

```
match('scene.pgm', 'book.pgm')
```

Code 4.2 MATLAB command to match using SIFT

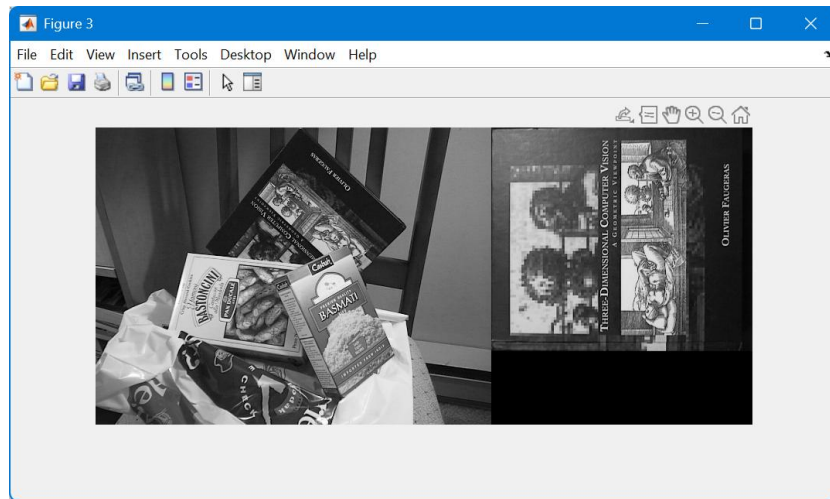


Figure 4.3 appendimages output before match

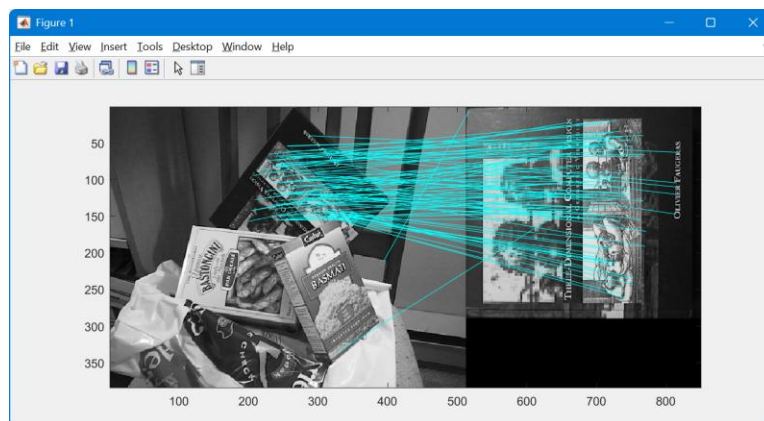


Figure 4.4 appendimages output with SIFT matching

4.3 C Examples

The `hello_sift` application was adapted from the course provided `sift.cpp`. This application identifies and circles the keypoints of the input image as seen in Figure 4.5 and Figure 4.6.



Figure 4.5 hello_sift features on snapshot_left_1385866691881.1577.jpg



Figure 4.6 hello_sift features on snapshot_right_1385866691881.1580.jpg

Next, we can use the `descriptor_extractor_matcher` application to match features of two images. In the first example, described by Code 4.3 and Figure 4.7, only the left snapshot is provided. The application manually rotates and modifies the scale of the image to demonstrate feature descriptor matching across scale and rotation invariance. You may also provide two input image arguments to apply the algorithm between the two instead of manually modifying the image, as described in Code 4.4 and Figure 4.8.

```
./descriptor_extractor_matcher SIFT SIFT BruteForce NoneFilter  
snapshot_left_1385866691881.1577.jpg 0
```

Code 4.3 Command to run `descriptor_extractor_matcher` SIFT on one snapshot

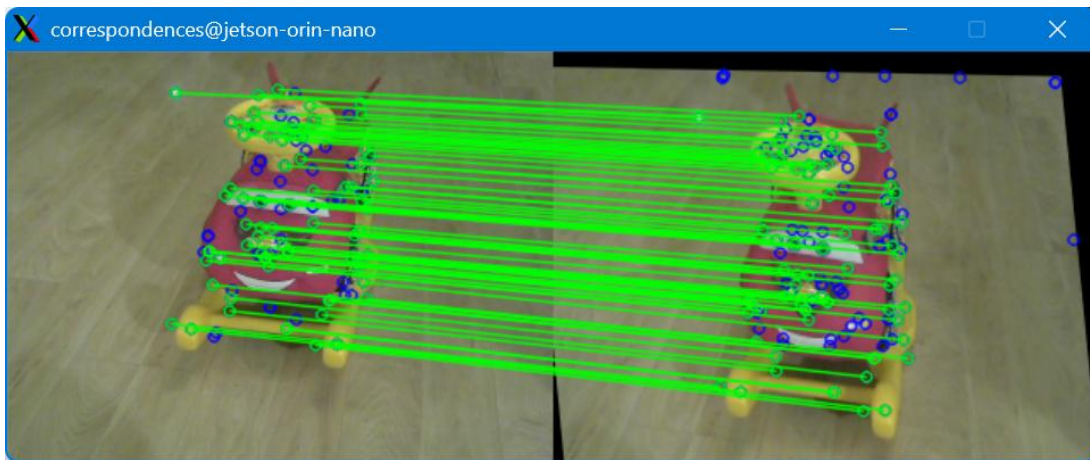


Figure 4.7 Correspondences of `descriptor_extractor_matcher` SIFT on one snapshot

```
./descriptor_extractor_matcher SIFT SIFT BruteForce NoneFilter  
snapshot_left_1385866691881.1577.jpg snapshot_right_1385866691881.1580.jpg 3
```

Code 4.4 Command to run `descriptor_extractor_matcher` SIFT on two snapshots



Figure 4.8 Correspondences of `descriptor_extractor_matcher` SIFT on two snapshots

Next, we move on to the siftDemoV4. This C application works very similarly to the MATLAB example described in Section 4.2 As seen below, the application uses the keys, generated from the sift MATLAB application, to match features between two input images.

```
./match -im1 book.pgm -k1 book.key -im2 scene.pgm -k2 scene.key > result.pgm
```

Code 4.5 Command to run match using MATLAB sift output keys.



Figure 4.9 115 matches of book.pgm traced onto scene.pgm

Finally, we will explore the ORB algorithm in comparison to SIFT. The descriptor_extractor_matcher application allows us to specify the ORB algorithm instead of the SIFT algorithm as seen in Code 4.6. My initial observation is that the algorithm identifies much more features within a clustered region than SIFT, but lacks some features identified on the edges of the object. We can see that there are much more concentrated green lines in the center of Figure 4.10 but there lacks features identified in the top right corner of the car and matches on the bottom left corner of the car as identified by the SIFT algorithm and shown above in Figure 4.8.

```
./descriptor_extractor_matcher ORB ORB BruteForce NoneFilter  
snapshot_left_1385866691881.1577.jpg snapshot_left_1385875341982.3462.jpg 3
```

Code 4.6 Command to run descriptor_extractor_matcher ORB on two snapshots



Figure 4.10 Correspondences of descriptor_extractor_matcher ORB on two snapshots

We can also apply ORB to one snapshot, internally modifying the image, similar to the example shown in Figure 4.7. Again, we see much more lines indicating matches but again lacking some on the edges of the image. I also notice there are less false descriptors identified using this algorithm. This is interesting given that SIFT is inherently more robust.

```
./descriptor_extractor_matcher ORB ORB BruteForce NoneFilter
snapshot_left_1385866691881.1577.jpg 3
```

Code 4.7 Command to run descriptor_extractor_matcher ORB on one snapshot



Figure 4.11 Correspondences of descriptor_extractor_matcher ORB on one snapshot

4.4 SIFT vs ORB

ORB, Oriented Fast Rotated Brief, is an OpenCV Labs developed algorithm that provides a more efficient and available alternative to SIFT. ORB was developed as a free alternative to SIFT, which was previously subject to patent restrictions, and is ideal for embedded systems because of the reduced computational cost. ORB differs from SIFT because it trades some robustness for performance. ORB uses Feature from Accelerated Segment Test (FAST) to find corners. But, this method doesn't handle scale or orientation, so ORB applies a Pyramidal Transform (for scale), as further explored in Section 4.5, and identifies centroids by intensity (for orientation). ORB also uses Binary Robust Independent Elementary Features (BRIEF) to compare pixels as binary strings. The orientation step mentioned before is critical for BRIEF to work properly.

Feature	SIFT	ORB
Scale invariant	Excellent	Good
Rotation invariant	Excellent	Very Good
Speed	Slow	Fast
Memory	Large	Small
Descriptor	Float (128D)	Binary
Matching	Euclidean distance	Hamming distance
Patent	Expired (previously patented)	Open Source
Embedded systems	Moderate	Excellent

In my opinion, ORB is the better algorithm for most applications, especially for embedded applications. The performance optimizations make the algorithm two orders of magnitude faster than SIFT. This is critical for real-time embedded applications that require low latency. Another reason ORB is advantageous is the low memory requirements. By using binary descriptors, as opposed to floating-point vectors, ORB is better suited for embedded systems that have memory constraints. Although lacking some of the accuracy of SIFT, ORB still applies very good feature detection that is more than enough for most applications. The algorithm makes real-time feature detection possible without significant hardware acceleration.

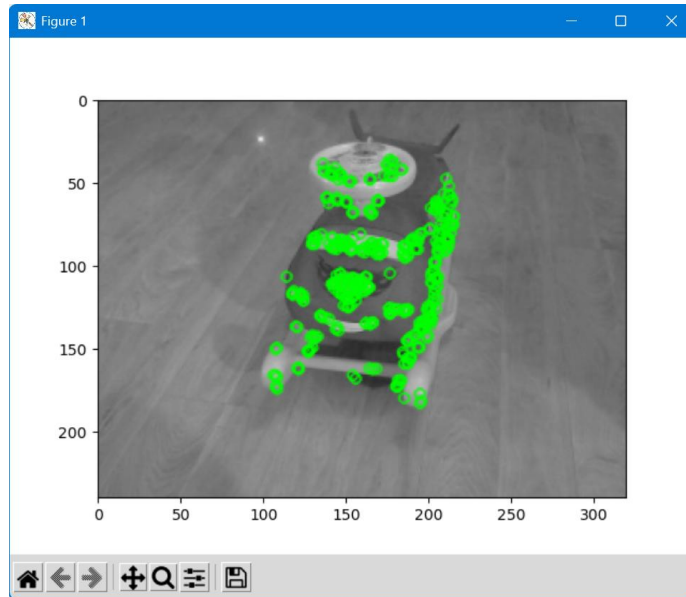


Figure 4.12 ORB tutorial for snapshot_left_1385866691881.1577.jpg

4.5 Pyramidal Transform

The pyramidal transform demo allows the user to zoom in and out of the input image. This shows a key step in the SIFT algorithm, generating scale space. By repeatedly subsampling the image to smaller resolutions, and then applying a blur at each of those levels, removing the finer details, the algorithm identifies a broader structure of the object. The goal of this is to achieve scale invariance. The multiple resolutions of the image combined with varying blurs isolate the key points that remain recognizable when the object is varying sizes. Figure 4.13 shows the original snapshot image. After zooming out, using the 'O' key, and then zooming in, using the 'I' key, Figure 4.14 and Figure 4.15 were created. Zooming out causes the image to half its resolution; zooming in doubles it. They showcase the different levels of subsample and blur that together create the scale space.

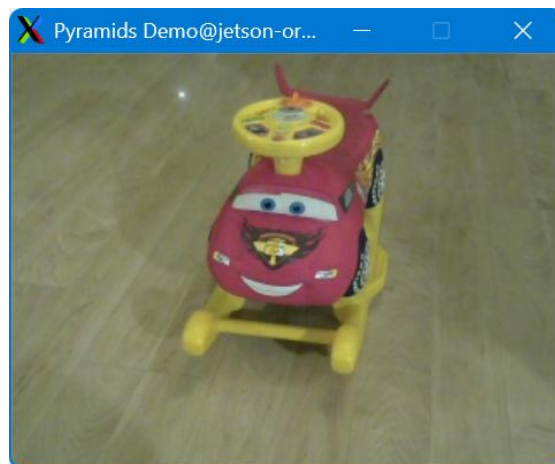


Figure 4.13 Original snapshot image

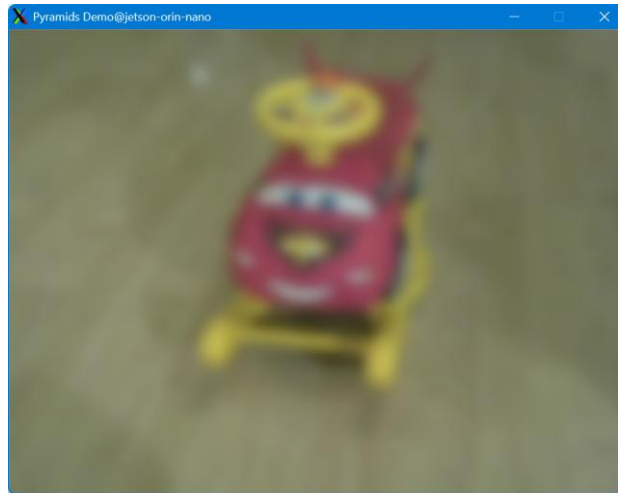


Figure 4.14 Modified snapshot image after two (2) zoom-out and two (2) zoom-in

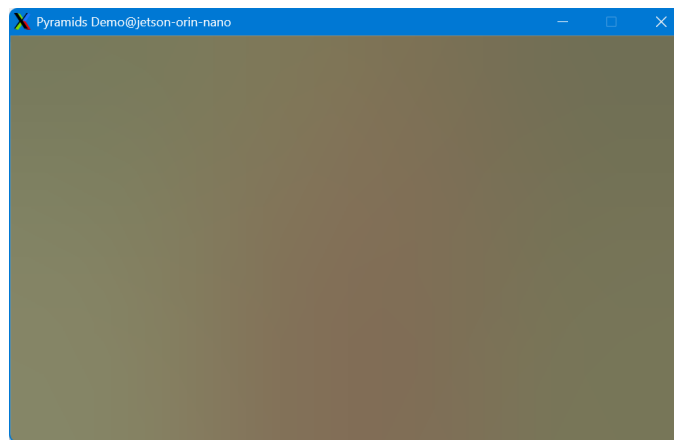


Figure 4.15 Modified snapshot image after five (5) zoom-out and five (5) zoom-in

While I was experimenting with the application, I observed a run-time constraint. If you zoom-in on the image too much, the application will quit due to an Insufficient memory error, as seen in Figure 4.16. This is because the image has become too big to be stored in RAM.

```
mason@jetson-orin-nano:~/ECV_5763/ex4/q4$ ./pyramids_snapshot_left_1385866691881.1577.jpg

Zoom In-Out demo
-----
* [i] -> Zoom in
* [o] -> Zoom out
* [ESC] -> Close program

** Zoom Out: Image / 2
** Zoom In: Image x 2
** Zoom In: Image x 2
** Zoom In: Image x 2
** Zoom In: Image x 2
** Zoom In: Image x 2
** Zoom In: Image x 2
** Zoom In: Image x 2
** Zoom In: Image x 2
** Zoom In: Image x 2
terminate called after throwing an instance of 'cv::Exception'
what():  OpenCV(4.8.0) /home/ubuntu/opencv_build/opencv/modules/core/src/alloc.cpp:73: error: (-4:Insufficient memory)
Failed to allocate 15099494400 bytes in function 'OutOfMemoryError'

Aborted (core dumped)
mason@jetson-orin-nano:~/ECV_5763/ex4/q4$
```

Figure 4.16 Insufficient memory error terminal output

OPTION #4 – Deeper Dive on CUDA

5 Basic CUDA

Build and run the basic NVIDIA CUDA example code and pick 3 applications to evaluate. Report your results showing CPU loading with “htop” and GP-GPU loading with “jtop.” Do any of the 3 applications you have selected saturate the GP-GPU and if not, why not?

First, I needed to reinstall OpenCV with CUDA support enabled. After investigating the reference material, I used the script [buildOpenCV.sh](#) to install OpenCV 4.10.0 with CUDA. I was able to install this relatively quickly by creating a large swap file and enabling parallel processing. After reinstalling OpenCV and installing jtop, this was my build configuration. I noticed right away that jtop did a good enough job showing CPU loading to

```
$ htop --version
htop 3.0.5
$ jtop --version
jtop 4.3.2
$ nvcc --version
nvcc: NVIDIA (R) Cuda compiler driver
Copyright (c) 2005-2024 NVIDIA Corporation
Built on Wed_Aug_14_10:14:07_PDT_2024
Cuda compilation tools, release 12.6, V12.6.68
Build cuda_12.6.r12.6/compiler.34714021_0
$ opencv_version
4.10.0
```

Code 5.1 Build Configuration

5.1 vectorAdd

The vectorAdd example adds two 50,000 element vectors element-by-element on the GPU. The application uses GPU threads to make each calculation. As seen in Figure 5.1, there is low GPU loading, around 25% utilization. The computation is simple and fast so the GPU finishes before any saturation. The CPU usage, as seen in Figure 5.2, is also low. Only one spike on CPU 6 and some usage on CPU 5 is seen. The CPU mainly allocates memory and copies data.

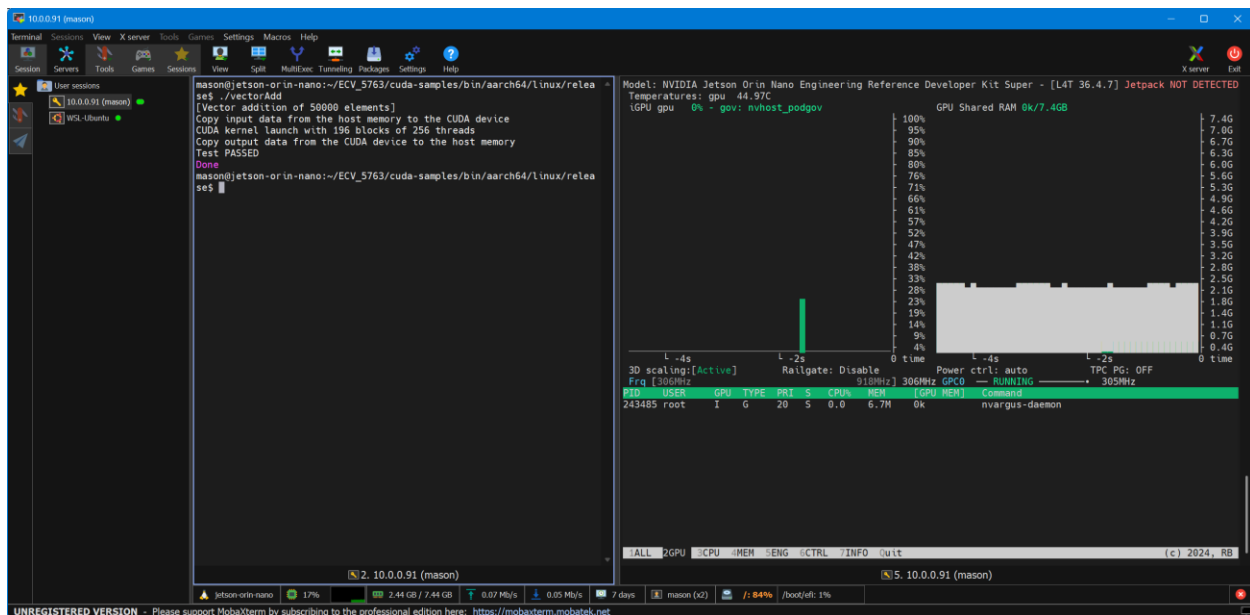


Figure 5.1 jtop capture for vectorAdd GPU loading

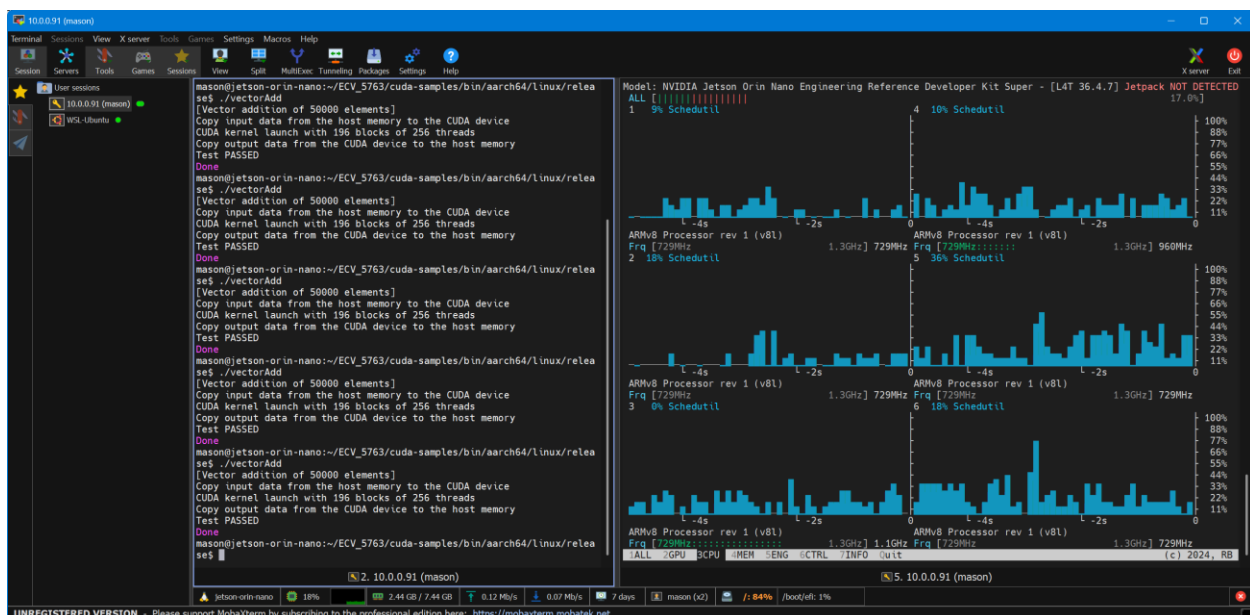


Figure 5.2 jtop capture for vectorAdd CPU loading

5.2 matrixMul

The matrixMul application demonstrates matrix multiplication using CUDA acceleration. The sample uses shared memory to access matrix tiles for multiplication. As seen in Figure 5.3, the GPU loading is high for a brief moment (less than half a second). The GPU appear to reach nearly 100% utilization indicating some saturation. This is due to the computationally intensive algorithm which GPU parallelism helps with. From the terminal output we see that the performance is 173.06 GFlops/s which is impressive for an embedded device. The CPU usage

(Figure 5.4) shows spiking on CPU 5 due to the memory allocation and copying but the other cores remain largely available.

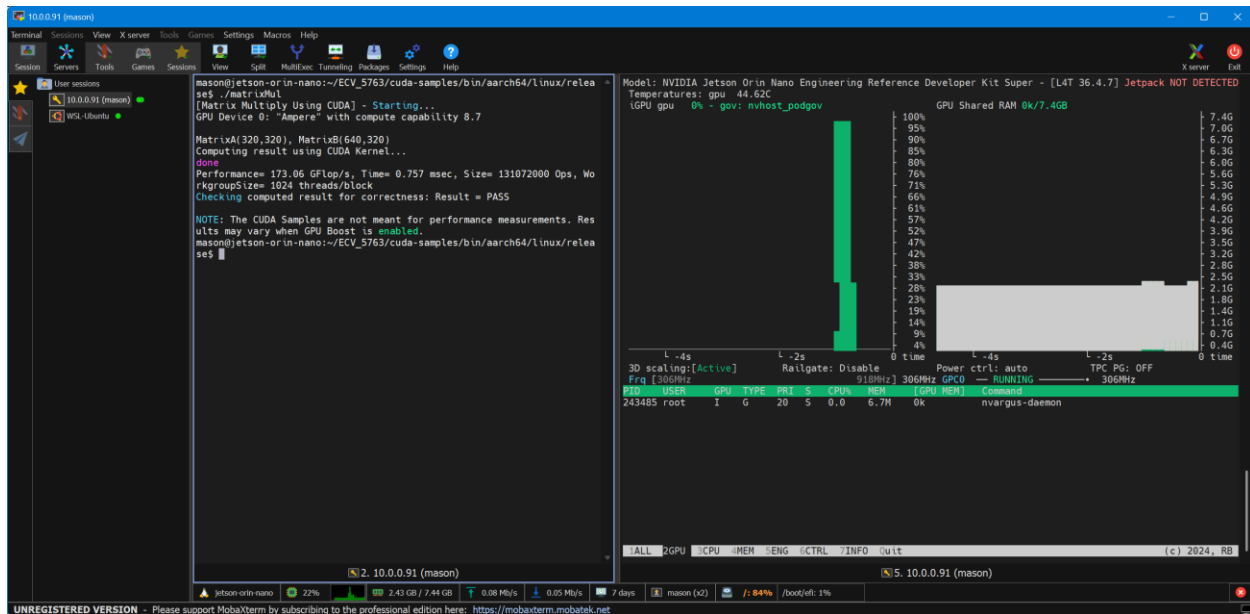


Figure 5.3 jtop capture for matrixMul GPU loading

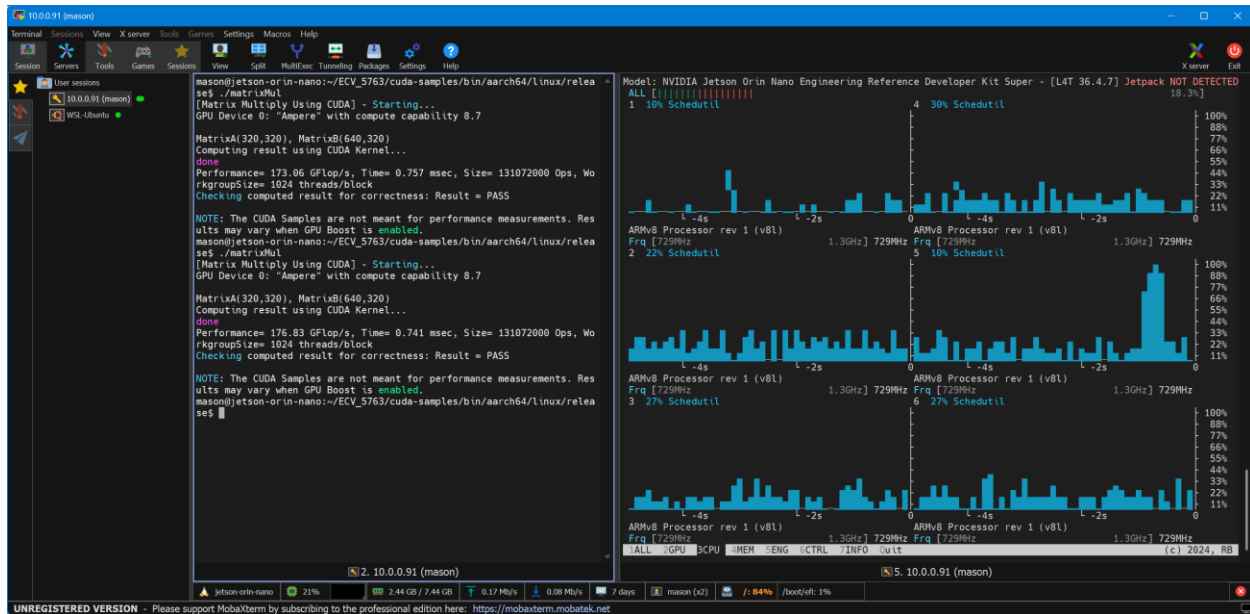


Figure 5.4 jtop capture for matrixMul CPU loading

5.3 bandwidthTest

The bandwidthTest application measures memory transfer bandwidth for host-device, device-host, and device-device memory copying. As demonstrated by the terminal output in Figure 5.5, the limiting factor for memory copying is the CPU. Device to device transfer (GPU to GPU) is

nearly three (3) times faster than transfer involving the host (CPU). In the figure, we see three (3) spikes of the GPU utilization for each transfer method test. It makes sense that the GPU would be saturated because we are conducting performance testing. The CPU remains largely available using only two cores, 4 and 6, heavily during the test.

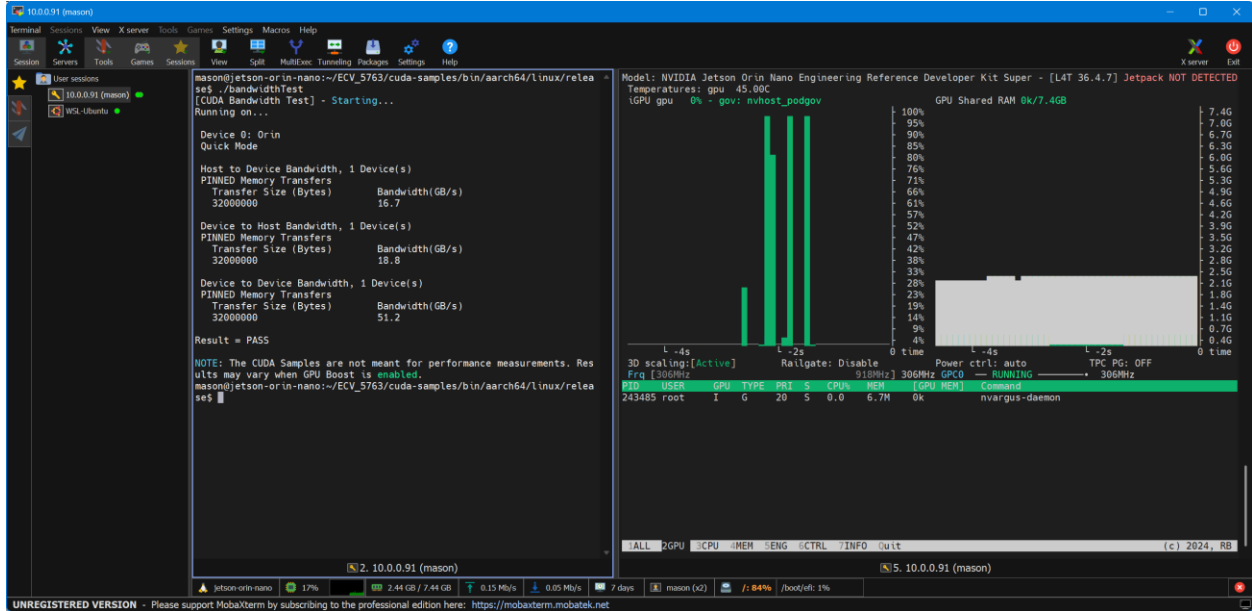


Figure 5.5 jtop capture for bandwidthTest GPU loading

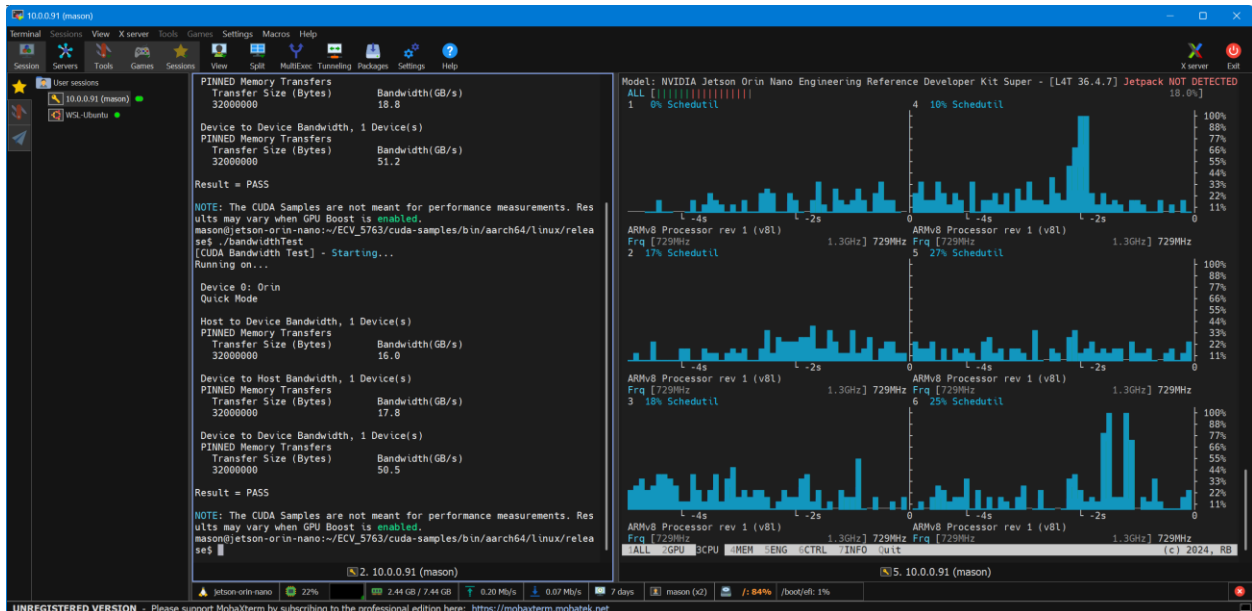


Figure 5.6 jtop capture for bandwidthTest CPU loading

5.4 simpleCudaGraphs

The simpleCudaGraphs application demonstrates how a sequence of GPU operations can be replayed efficiently to increase performance. As seen in Figure 5.7, there is brief GPU saturation during the height of the computation but it is very quick. The CPU usage, in Figure 5.8, shows saturation of core 1 and core 5.

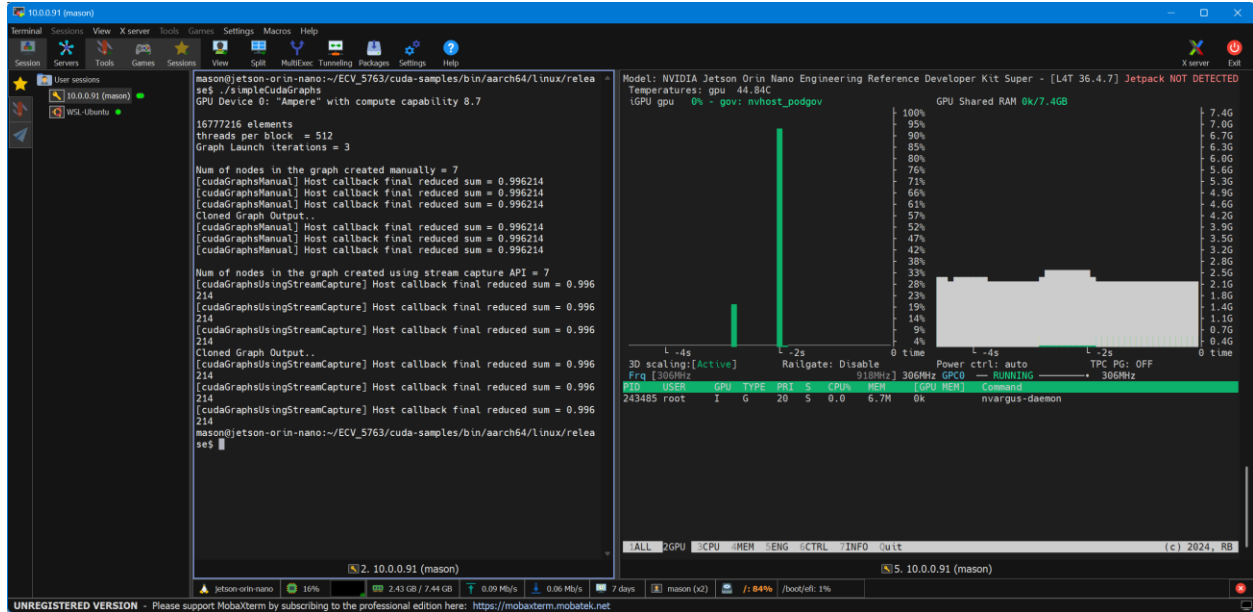


Figure 5.7 jtop capture for simpleCudaGraphs GPU loading

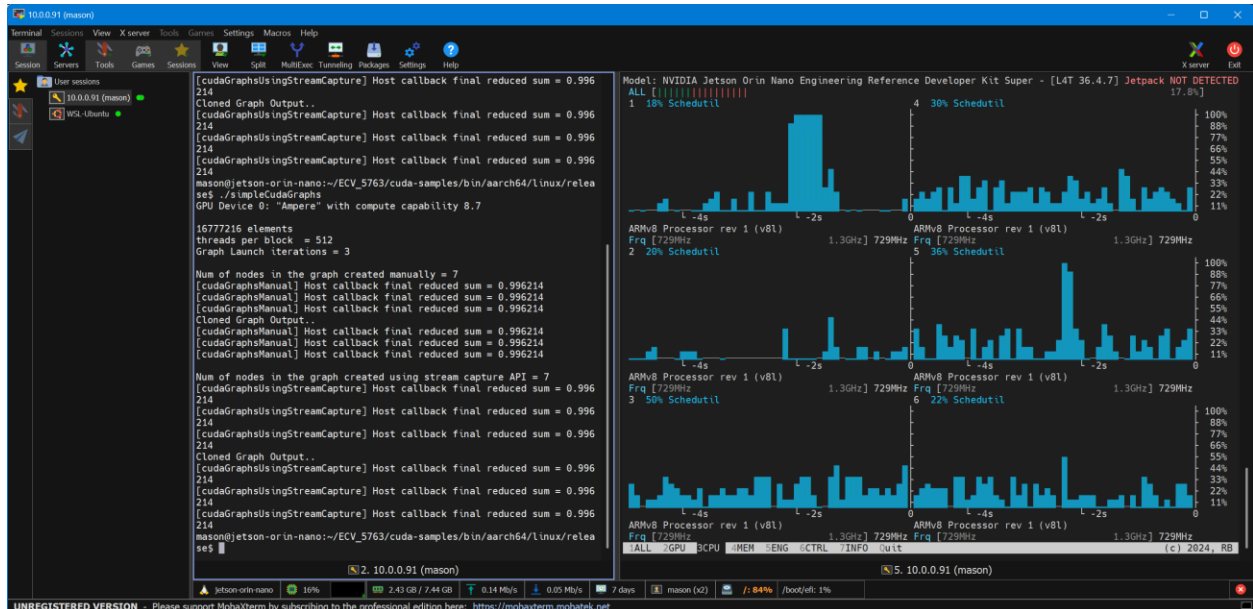


Figure 5.8 jtop capture for simpleCudaGraphs CPU loading

6 Cuda Interactive Hough

Create your own CUDA accelerated application for your centroid tracker from lab #3 or any other image transform you would like other than sharpen (e.g., Sobel, Laplace, DCT, etc.) [note that these examples I provide from previous labs may help - sharpenCUDA/ stand-alone, or as an 8th example - 8_ACV_CUDA_Examples/CUDA_vs_Mulit-Corecompare/]. Show that your CUDA code works and that it is running on the GP-GPU by showing loading with “jtop.”

First, I observed the resting state of the GPU showing, as expected, 0% utilization, and about 2.3G of shared RAM.

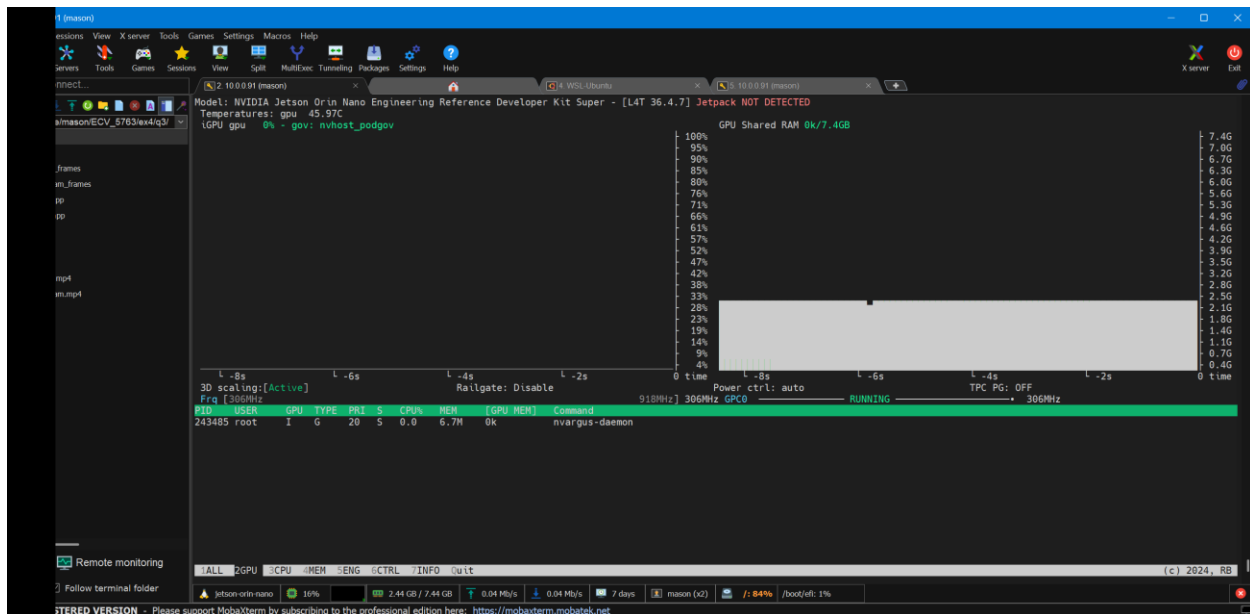


Figure 6.1 jtop capture for initial state GPU loading

6.1 Static Image

First, I implemented the application using a static image. This made debugging easier. The image is only processed once by the GPU to grayscale and on state changes to Hough detection modes. Because this happens so quickly it was difficult to capture the GPU loading using jtop. For all the state changes using Hough detection methods, including parameter changes made on the trackbar, there was one spike of low GPU usage, as seen in Figure 6.2.

Figure 6.3, Figure 6.4, and Figure 6.5 show CUDA Hough circle detector, line detector (standard Hough line transform), and segment detector (probabilistic Hough line transform), respectively.

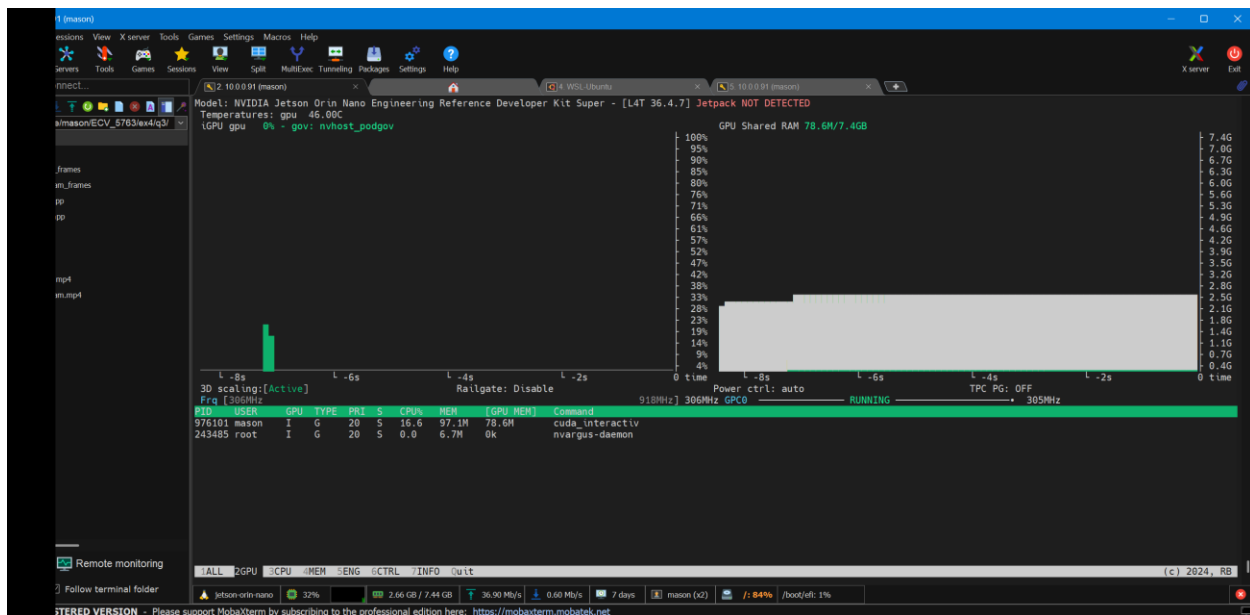


Figure 6.2 jtop capture for cuda_interactive_hough GPU loading – static image

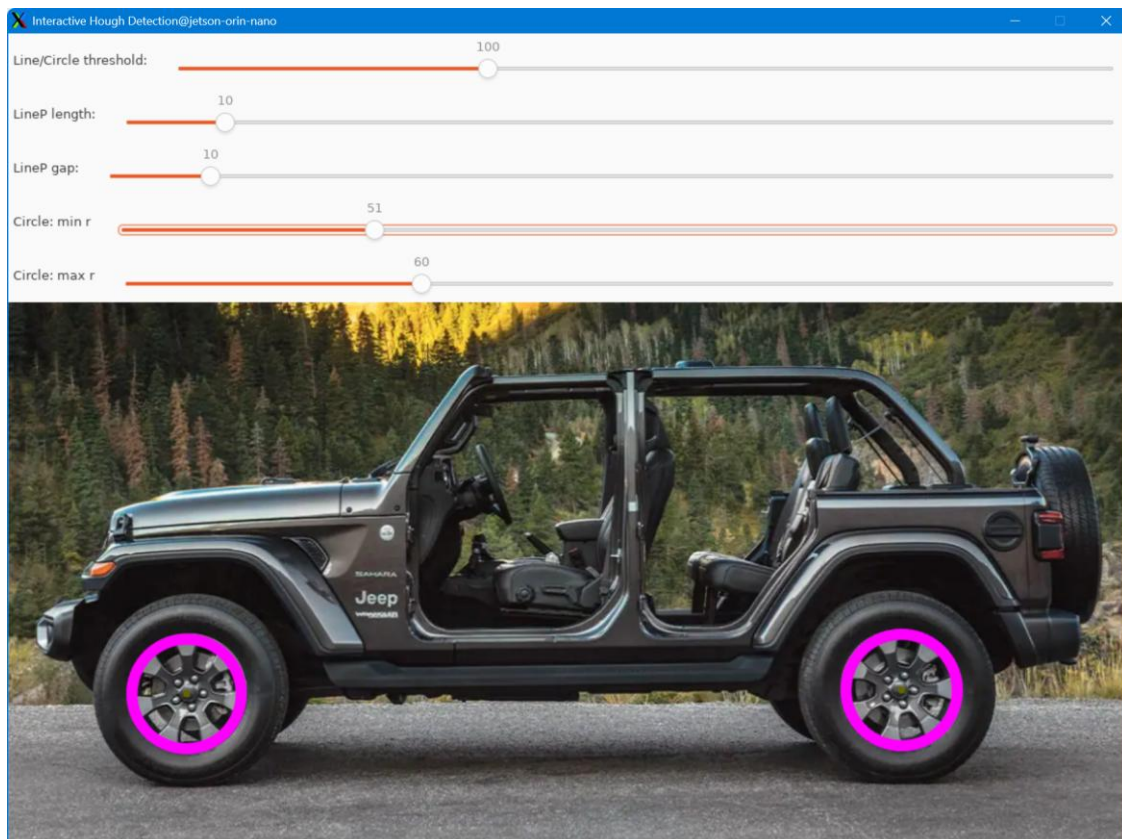


Figure 6.3 CUDA circle detector on Jeep-Sideview.png

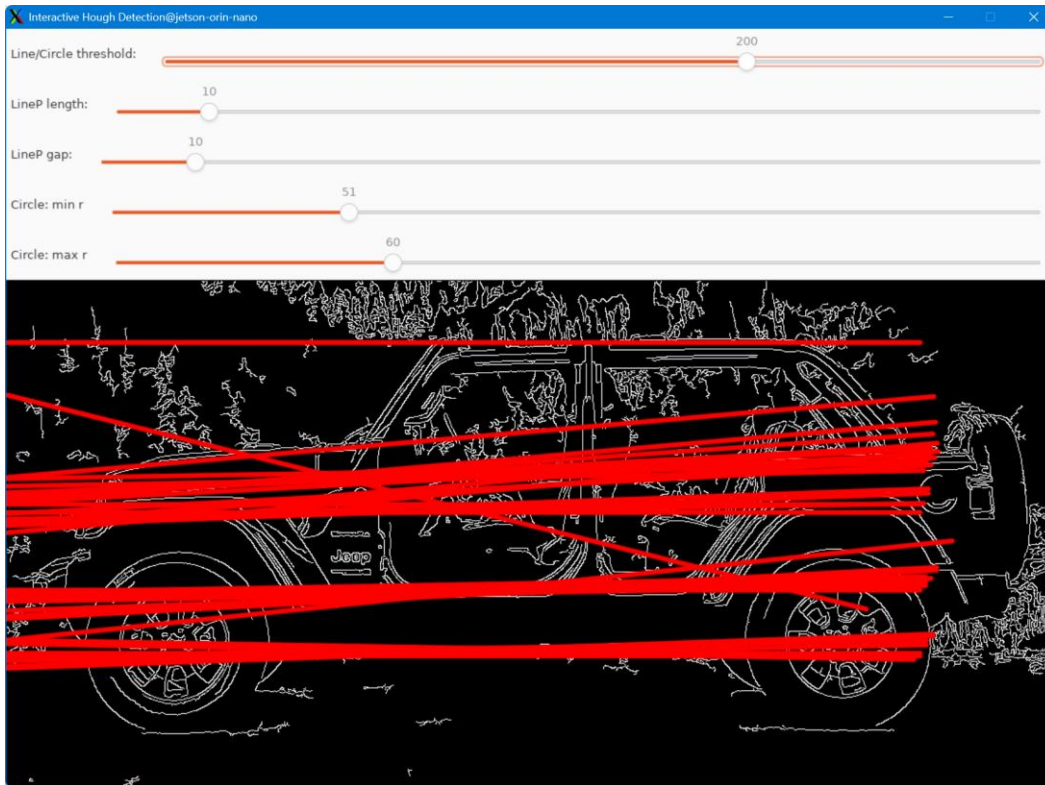


Figure 6.4 CUDA hough line detector on Jeep-Sideview.png

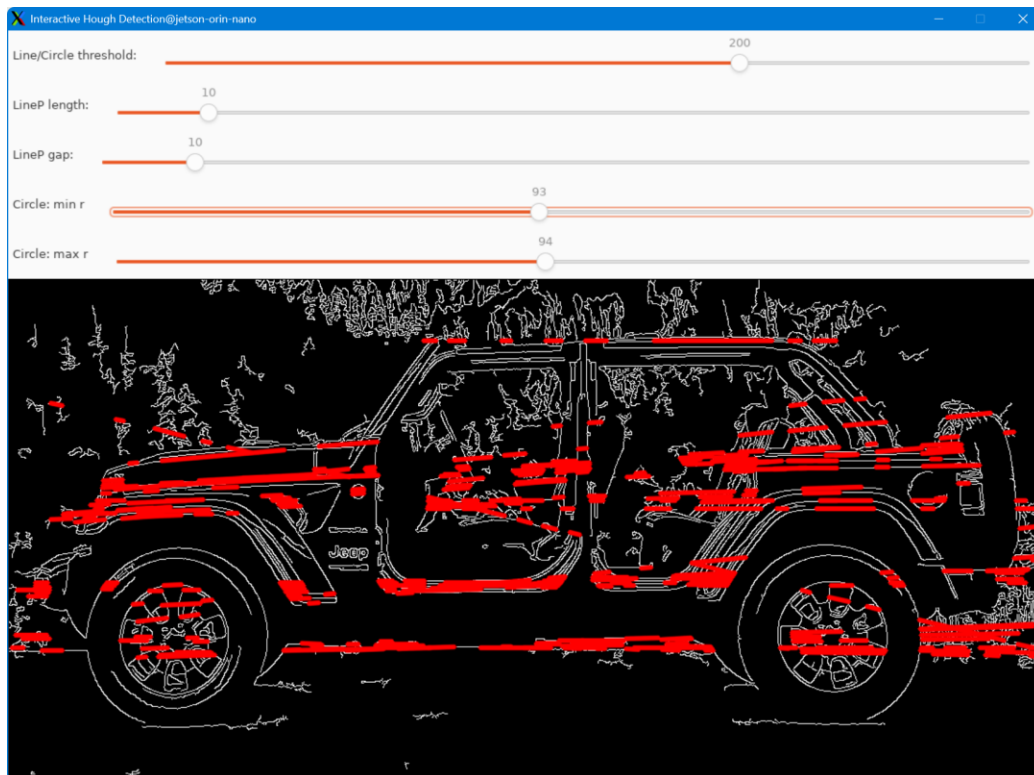


Figure 6.5 CUDA hough segment detector on Jeep-Sideview.png

6.2 Live Webcam

After getting a working application using the static image, I implemented the live webcam feature. With this configuration the GPU is used more but it doesn't ever become saturated to the relatively low amount of computational processing required for this application.

Probabilistic Hough line detection was implemented using the CUDA HoughSegmentDetector. The output GUI seen in Figure 6.6 and caused the corresponding GPU utilization shown in Figure 6.7. As you can see, every frame captured is processed using the OpenCV CUDA functions which is why we see frequent moderate GPU usage.

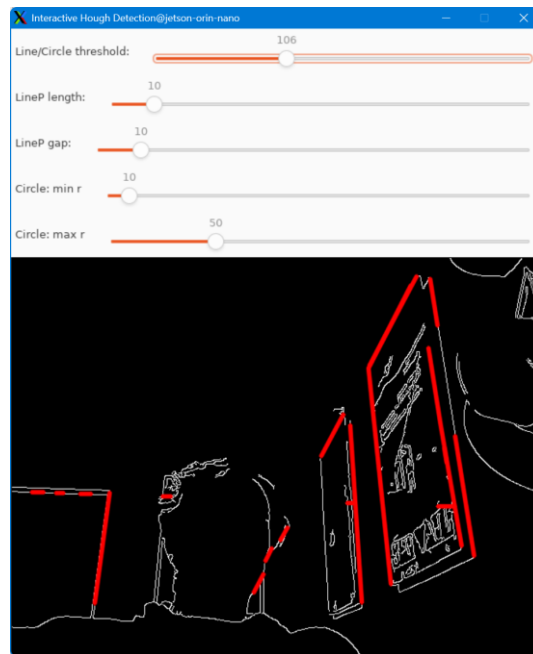


Figure 6.6 CUDA hough segment detector on webcam capture

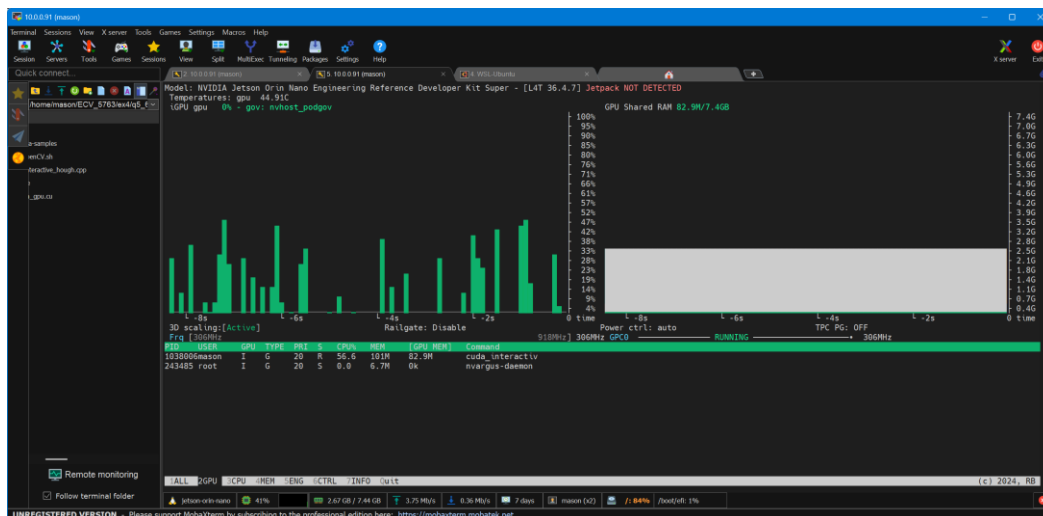


Figure 6.7 jtop capture for cuda_interactive_hough GPU loading – live segment detector

Standard Hough line detection was implemented using the CUDA HoughLineDetector. The output the GUI seen in Figure 6.8, with lines extending to the border of the image. As demonstrated in Figure 6.9, GPU usage is similar to probabilistic detection.

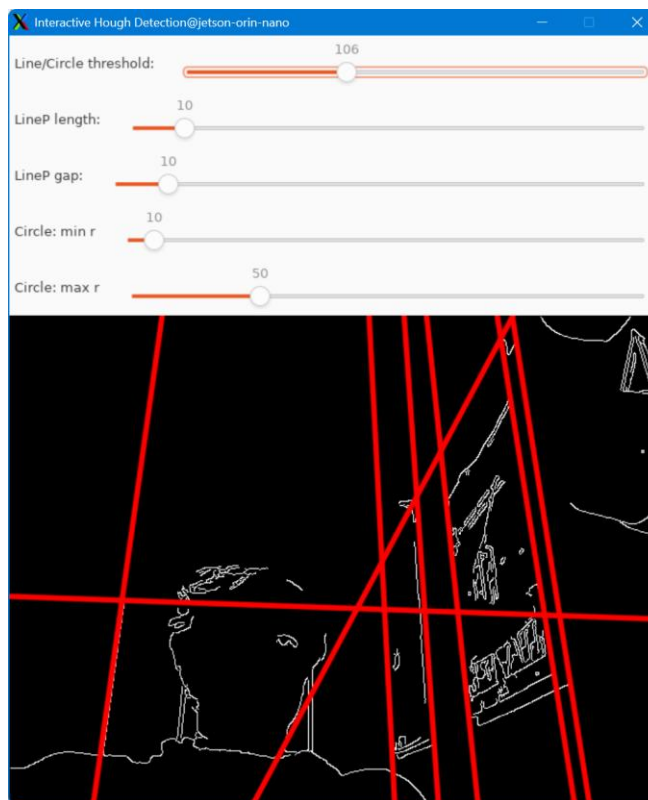


Figure 6.8 CUDA hough line detector on webcam capture

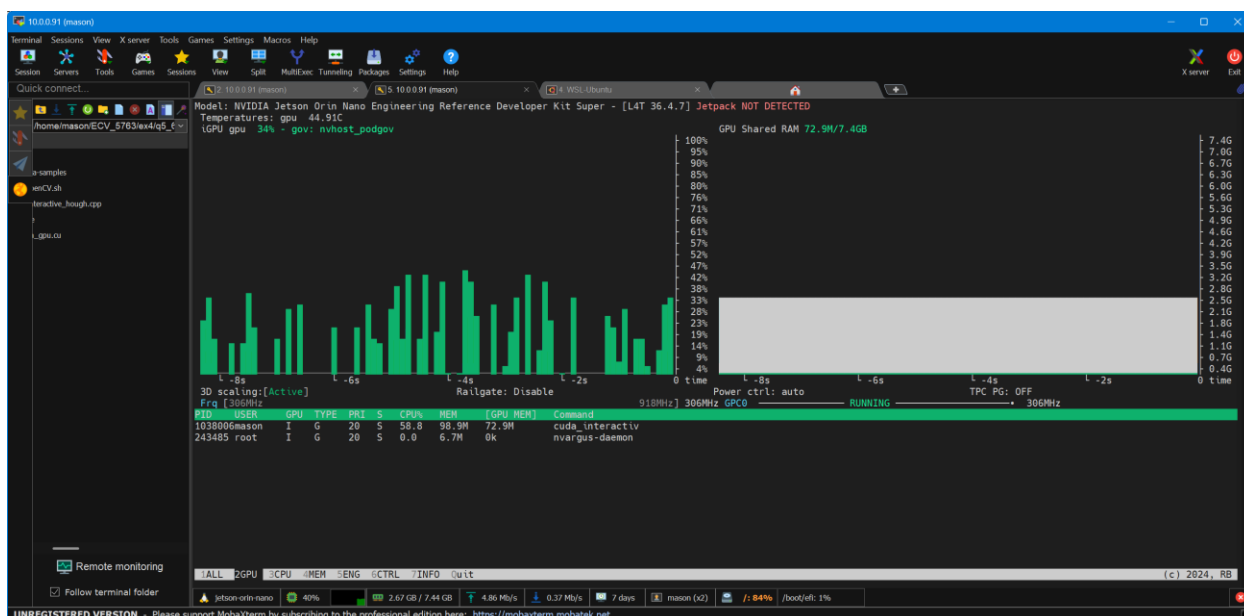


Figure 6.9 jtop capture for cuda_interactive_hough GPU loading – live line detector

Hough circle detection was implemented using the CUDA HoughCirclesDetector. As seen in Figure 6.10, below, the application correctly identifies the coin that I am holding as a circle. The circle detection had slightly lower GPU usage, as shown in Figure 6.11, but overall consistent usage corresponding with each processed frame.

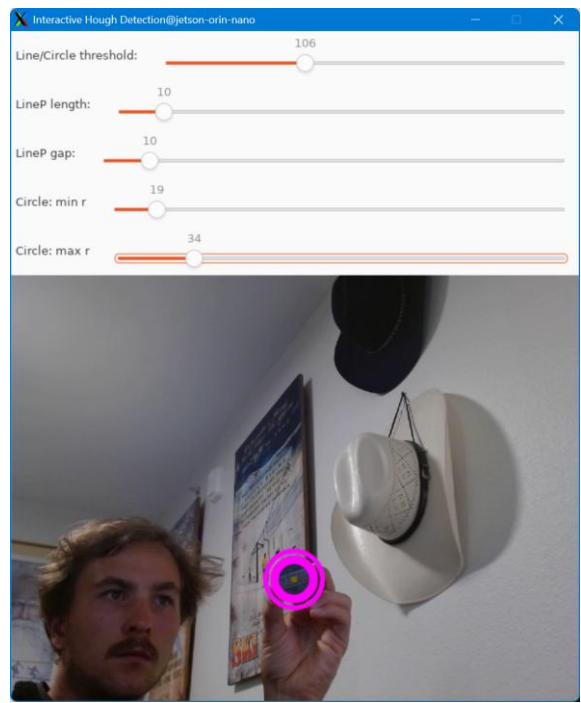


Figure 6.10 CUDA Hough circle detector on webcam capture

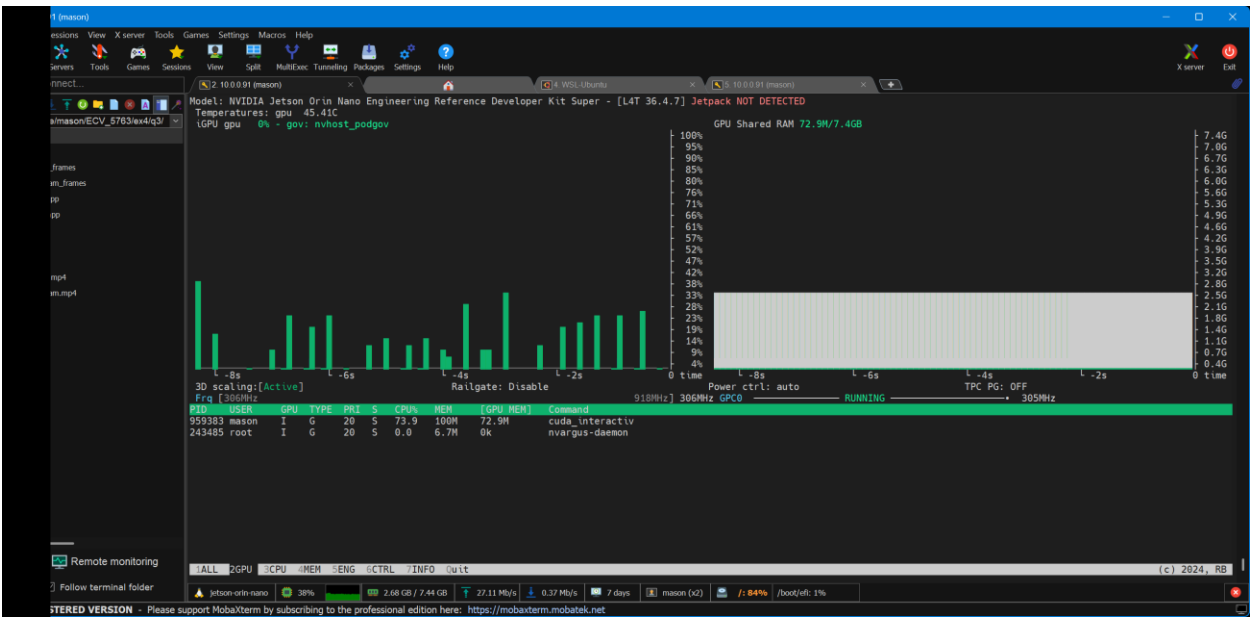


Figure 6.11 jtop capture for cuda_interactive_hough GPU loading – live circle detector

The last states observed were regular and grayscale mode. These states caused significantly less GPU usage, as seen in Figure 6.12, because the Canny and Hough detectors were not computing. However, each frame processed was converted to grayscale and blurred using CUDA methods which is why there is still some (very little) GPU usage.

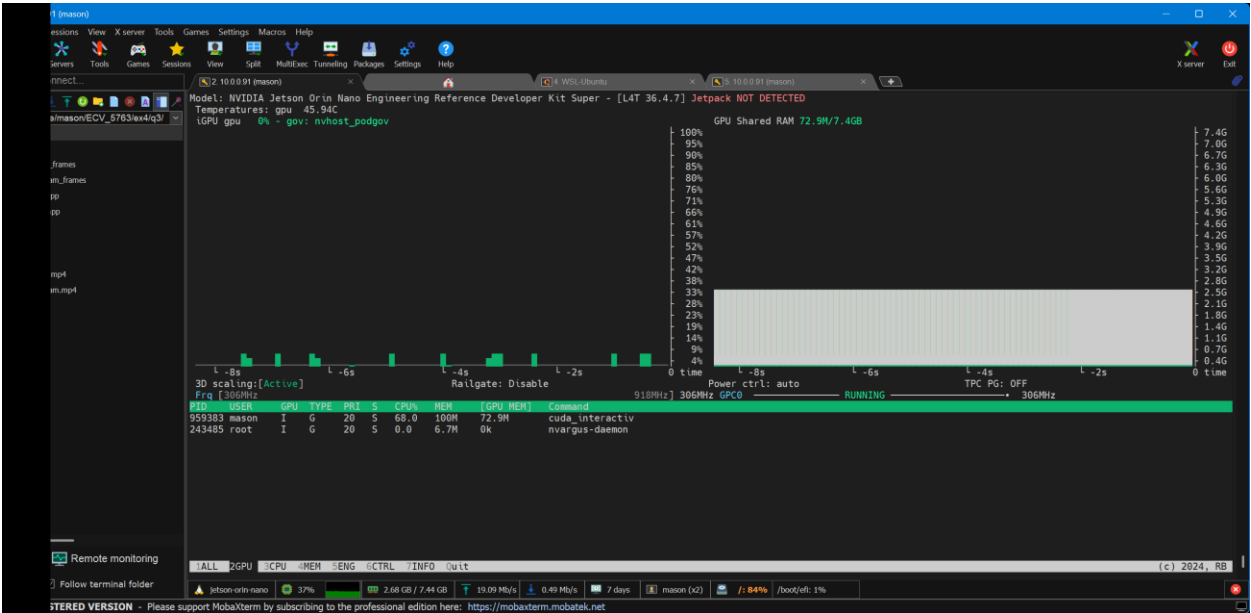


Figure 6.12 jtop capture for cuda_interactive_hough GPU loading – live regular/grayscale mode

7 References

- [1] OpenCV, "OpenCV 4.8.0 Documentation." [Online]. Available: <https://docs.opencv.org/4.8.0/index.html>.
- [2] R. O. Duda and P. E. Hart, "Use of the Hough transformation to detect lines and curves in pictures," *Commun. ACM*, vol. 15, no. 1, pp. 11–15, Jan. 1972, doi: 10.1145/361237.361242.
- [3] D. G. Lowe, "Distinctive Image Features from Scale-Invariant Keypoints," *Int. J. Comput. Vis.*, vol. 60, no. 1, pp. 91–110, Jan. 2004, doi: 10.1023/B:VISI.0000029664.99615.94.
- [4] D. Lowe, "The SIFT Keypoint Detector," *University of British Columbia Computer Science Department*, 2004. [Online]. Available: <https://www.cs.ubc.ca/%7Elowe/keypoints/>
- [5] OpenCV Content Team, "ORB (Oriented FAST and Rotated BRIEF)," *OpenCV 4.8.0 Documentation*, 2023. [Online]. Available: https://docs.opencv.org/4.8.0/d1/d89/tutorial_py_orb.html
- [6] OpenCV Content Team, "Image Pyramids," *OpenCV 4.8.0 Documentation*, 2023. [Online]. Available: https://docs.opencv.org/4.8.0/d4/d1f/tutorial_pyramids.html
- [7] JetsonHacks, "OpenCV 4 + CUDA on Jetson Nano," *JetsonHacks*, Nov. 22, 2019. [Online]. Available: <https://jetsonhacks.com/2019/11/22/opencv-4-cuda-on-jetson-nano/>
- [8] S. Siewert, "Tutorial: CUDA and Multi-Core," *ECV-5763-CU Lecture Documents*, ESEE Web Resources, University of Colorado Boulder. [Online]. Available: <https://o365coloradoedu-my.sharepoint.com/>
- [9] R. Bonghi, "jetson_stats," GitHub repository. [Online]. Available: https://github.com/rbonghi/jetson_stats
- [10] M. Degans, "build_opencv.sh," *nano_build_opencv*, GitHub. [Online]. Available: https://github.com/mdegans/nano_build_opencv/blob/master/build_opencv.sh
- [11] OpenCV, "OpenCV 4.10.0 Documentation." 2024 [Online]. Available: <https://docs.opencv.org/4.10.0/>
- [12] NVIDIA Corporation, "CUDA Samples," GitHub repository. [Online]. Available: <https://github.com/NVIDIA/cuda-samples/tree/master>

8 Appendix

GitHub repository: https://github.com/mamc3334/ECV_5763/tree/ex4

Pull request: https://github.com/mamc3334/ECV_5763/pull/5

Forked ECV GitHub repository: <https://github.com/mamc3334/ECV-ECEE-5763-mcgaffin>

Pull request: N/A